Normalidade Contextual em APIs: Caracterização de Regimes a partir de Access Logs para Apoio à Análise de Incidentes

Diego Paz Casagrande
Universidade de Brasília
Brasília, DF, Brasil

ABSTRACT

Quando um incidente é aberto, uma das primeiras perguntas do operador é se as APIs envolvidas estão operando normalmente e, se não estão, qual indicador saiu do padrão. A resposta não é trivial, porque o comportamento de uma API varia legitimamente com o horário, o calendário e os processamentos em lote, e muda ao longo dos meses. Este trabalho investiga se API Access Logs (AALs) agregados por minuto permitem responder, no instante do registro do incidente, se cada endpoint de API está em um de seus regimes normais (Normal) ou fora deles (Residual); qual indicador é responsável: erros de negócio (4xx), erros de sistema (5xx) ou latência; e desde quando. Seguindo o CRISP-DM, propõe-se um perfil de normalidade contextual por endpoint, com um limite próprio para cada indicador calibrado por validação cruzada, e uma expansão retroativa em faixas de 15min a 4h para estimar o início do desvio. Em 5,6 milhões de registros de 12 endpoints de produção no período de maio a setembro de 2026, testes estatísticos confirmaram regimes ligados à hora do dia. Em 12.157 consultas simuladas em agosto e setembro, a decisão sobre os 15 minutos anteriores ao registro teve especificidade de 97,1% e apontou o indicador correto em 96,7% dos casos detectados; o início real ficou dentro da zona estimada em 91,9% deles. A sensibilidade foi de 39%: alta para erros de sistema (até 87%), baixa para erros de negócio e, para latência, de 34% a 64% nas degradações de 4 e 10 vezes. Um histórico de 1 mês superou o de 3 meses, indicando que o comportamento das APIs muda ao longo do tempo e que o retreino frequente é preferível a um histórico longo.

PALAVRAS-CHAVE

Mineração de Dados; API Access Log; Normalidade Contextual; Análise de Incidentes; CRISP-DM; Regimes Operacionais.

1 INTRODUÇÃO

A arquitetura dos sistemas modernos consolidou o uso de APIs (*Application Programming Interfaces*) como mecanismo fundamental de integração em sistemas distribuídos. Em ambientes corporativos de grande porte, as APIs medeiam desde requisições de clientes externos até transações entre sistemas legados e modernos. No Banco do Brasil, cerca de 30 mil APIs são executadas diariamente, conectando soluções de negócio que combinam plataformas em nuvem e *mainframe*.

1.1 Contexto

Requisições processadas por gateways de API geram registros com o instante da execução, o recurso acessado, o código de resposta, o tempo de processamento e o consumidor. Neste estudo, esses registros são denominados *API Access Logs* (AALs), o registro gerado pelo gateway para as requisições processadas, distinto do *execution log* e do log de aplicação. Os AALs estão disponíveis de forma padronizada para um conjunto amplo de APIs, sem instrumentação específica no código.

O comportamento operacional de uma API não é necessariamente estacionário. Volume, latência e proporção de erros variam com o horário, o dia da semana, os processamentos em lote e as janelas de manutenção. Uma taxa de erros elevada pode ser normal na virada do dia, assim como latência maior pode indicar degradação ou apenas um aumento esperado de volume de requisições; uma queda abrupta de tráfego pode indicar indisponibilidade ou o fim de um período de atendimento.

1.2 Problema

Quando um incidente é registrado, o operador precisa contextualizar semanticamente a situação e responder se para cada API envolvida: *o período anterior ao registro estava dentro do comportamento normal esperado para aquele momento?* Se não estava, também importa saber quais métricas desviaram, como: erros de negócio (faixa 4xx), erros de sistema (faixa 5xx) ou latência; e desde quando o desvio acontece. Como informação complementar, interessa saber se o impacto é generalizado, ou concentrado em uma aplicação cliente específica com volume suficiente para alterar o estado da API, um erro espalhado entre todos os consumidores sugere um problema no provedor.

Hoje essa resposta depende de consultar várias ferramentas e de conhecer o comportamento típico de cada API e endpoint [**?**]. Abordagens com limites fixos ou com um único perfil de normalidade não distinguem a variação legítima, como um lote à meia-noite ou o volume menor no domingo, de um desvio real.

Do ponto de vista de mineração de dados, o problema consiste em: (1) caracterizar os regimes normais de cada recurso a partir de AALs agregados em janelas; (2) classificar o período imediatamente anterior ao registro de um incidente em dois estados: Normal (dentro de algum regime normal esperado para aquele contexto) ou Residual (todo o resto); e (3) quando Residual, identificar qual parâmetro desviou, sendo este o indicador responsável.

1.3 Motivação

Este trabalho complementa um projeto de mestrado sobre sistemas multiagente para apoio ao atendimento de incidentes. A literatura recente mostra agentes especializados em diferentes fontes de dados, coordenados para triagem e análise [**???**]. Um desses agentes precisa de uma fonte confiável e explicável sobre o comportamento das APIs. O foco não é monitoração em tempo real nem a identificação da causa raiz, mas a *informação de contexto no instante do incidente*: dizer se cada endpoint está em regime normal e, se não, qual indicador está impactado e desde quando. Essa informação abastece o operador por meio de um agente especialista e colabora para a análise do ocorrido entregando um contexto semântico embasado nas atividades da mineração de dados, economizando recursos computacionais e evitando alucinações no agente dada a quantidade de informações necessárias à análise.

1.4 Objetivos

Objetivo geral: caracterizar os regimes normais de cada endpoint da API a partir de Access Logs agregados por minuto e no instante do registro de um incidente, informar se o endpoint está em regime NORMAL e, se não, qual indicador foi impactado e desde quando.

Objetivos específicos:

- Verificar se cada endpoint apresenta vários regimes normais ligados ao contexto (hora e tipo de dia).
- Construir um perfil de normalidade por endpoint e contexto e uma regra de decisão NORMAL ou RESIDUAL explicável.
- Identificar o indicador responsável (4xx, 5xx, latência) e estimar o início do desvio.
- Avaliar o método em cenários de incidente simulados em dois meses e compará-lo a baselines simples.
- Avaliar quanto histórico o perfil precisa (3 meses ou 1 mês).
- Disponibilizar a informação em formato estruturado para um agente ou operador.

1.5 Questões de pesquisa

- **RQ1.** AALs agregados permitem caracterizar os regimes normais de cada recurso e decidir, no instante do registro, se o recurso está em regime NORMAL?
- **RQ2.** Quando o recurso não está em regime NORMAL, o método identifica corretamente o indicador responsável (4xx, 5xx, latência) e o início do desvio?
- **RQ3.** Qual o período anterior ao registro (15, 30 ou 60 min) mais adequado para a decisão, e como o intervalo entre o início da degradação e o registro afeta o resultado no regime NORMAL?
- **RQ4.** Um histórico mais longo (3 meses) produz um perfil melhor do que o histórico recente (1 mês)?

1.6 Contribuições e organização

As contribuições são: (i) a caracterização rastreável dos regimes de 12 recursos de API de produção ao longo de cinco meses; (ii) um método simples e explicável, sem rótulos, que decide o estado de um recurso no instante do registro e aponta o indicador impactado; (iii) uma estimativa de início por expansão retroativa; (iv) um protocolo de avaliação com cenários de incidente simulados em dois meses; e (v) evidência de que, nesses dados, o histórico recente é mais útil que o longo. A Seção **??** discute os trabalhos correlatos; a Seção **??**, a metodologia; a Seção **??**, os resultados; a Seção **??**, a discussão; a Seção **??**, as ameaças à validade; e a Seção **??** conclui.

2 TRABALHOS CORRELATOS

A literatura relacionada é organizada em quatro frentes e avaliada segundo quatro critérios que decorrem do problema tratado aqui: (i) a fonte de dados exigida; (ii) o tratamento dado à normalidade, isto é, se o comportamento esperado é único ou condicionado ao contexto; (iii) o tipo de saída produzida, escore ou explicação; e (iv) a necessidade de rótulos ou de instrumentação adicional. A Tabela **??** resume a comparação.

2.1 Análise e mineração de logs

? propuseram o DeepLog, que modela sequências de eventos de log com redes LSTM e sinaliza como anômala toda sequência pouco provável sob o modelo aprendido. **?** estenderam essa linha com o LogAnomaly, combinando a informação semântica dos *templates* com a detecção de anomalias sequenciais e quantitativas. Ambos sustentam a premissa que também adotamos: logs carregam informação suficiente para caracterizar comportamento, sem instrumentação dedicada.

As diferenças, porém, são decisivas para o problema aqui tratado. Primeiro, a unidade de análise é a sequência de eventos de log de sistema, que pressupõe *templates* estáveis e acesso ao log de aplicação; o AAL é um agregado por minuto, sem ordem de eventos e sem texto a partir do qual extrair *templates*. Segundo, e mais importante, a normalidade nesses modelos é única: aprende-se uma distribuição de sequências normais, e o que foge dela é anômalo. Um lote à meia-noite ou uma janela de manutenção produzem padrões legítimos que um modelo sem contexto tende a tratar como desvio, a menos que o horário entre implicitamente como atributo, o que os trabalhos não exploram. Terceiro, a saída é um escore ou um rótulo binário. Para o operador que acaba de abrir um incidente, saber que a sequência é improvável não responde qual indicador saiu do padrão nem desde quando.

2.2 Normalidade e séries temporais

? propuseram o SPOT e sua variante com *drift*, o DSPOT, que calibram limiares a partir da cauda da distribuição usando Teoria de Valores Extremos, com adaptação a séries não estacionárias. É o trabalho mais próximo do nosso em espírito: ambos reconhecem que o limiar não pode ser fixo e o derivam da própria distribuição observada, sem rótulos. O DSPOT adapta o limiar *ao longo do tempo*, seguindo a série; o perfil contextual a trata como função de variáveis conhecidas *a*

priori, hora e tipo de dia. A consequência prática é que a adaptação temporal absorve uma degradação lenta como se fosse o novo normal, ao passo que o condicionamento por contexto preserva a referência do mesmo horário em dias anteriores. Em compensação, o DSPOT não exige que os contextos relevantes sejam conhecidos de antemão, o que é uma vantagem em domínios onde a sazonalidade não é óbvia.

? propuseram o USAD, baseado em autoencoders para séries multivariadas. O método é mais expressivo que o nosso e captura dependências entre indicadores. Há ainda a questão do treinamento, que pressupõe um período limpo de anomalias; em dados de produção sem histórico de incidentes rotulado, como os nossos, essa suposição não é verificável.

? documentam a dificuldade de transferir modelos de detecção entre sistemas distintos, mesmo com meta-aprendizado. O achado é relevante como justificativa negativa para nossa escolha de ajustar um perfil por recurso se a transferência entre sistemas é difícil, a transferência entre recursos de perfis de tráfego muito diferentes nos nossos dados dificilmente seria mais fácil. A contrapartida é que perfis por recurso não generalizam para recursos novos, que precisam acumular histórico antes de serem avaliados; nossos resultados em setembro, com recursos de histórico curto, mostram esse custo.

2.3 Modelagem comportamental de microsserviços

O DeepTraLog [**?**] combina *traces* e logs em um grafo de eventos e aprende, com redes neurais em grafos, a estrutura esperada de uma requisição; o GAMMA [**?**] localiza gargalos de desempenho explorando a topologia de dependências entre serviços. Ambos entregam algo que o presente trabalho não entrega: localização, isto é, a indicação de *onde* na cadeia de chamadas o problema se manifesta.

Esse ganho tem um pré-requisito. Ambos pressupõem *traces* distribuídos ou métricas de infraestrutura correlacionadas, que exigem instrumentação consistente em todos os serviços da cadeia. Em ambientes corporativos heterogêneos, que combinam plataformas em nuvem e sistemas legados, essa cobertura é desigual. O AAL tem a propriedade oposta: é pobre em estrutura, porque não enxerga além do *gateway*, mas é uniformemente disponível. Os dois enfoques são, portanto, complementares, e não concorrentes.

2.4 Incidentes e sistemas multiagente

O Triangle [**?**] organiza múltiplos agentes para a triagem de incidentes; o OpsAgent [**?**] e o trabalho de **?** coordenam agentes especializados em diferentes fontes para análise de incidentes e determinação de causa raiz. Esses sistemas deslocam o problema. O desafio deixa de ser o algoritmo de detecção e passa a ser a qualidade e a legibilidade das evidências que cada agente consome.

É nesse ponto que a avaliação desses trabalhos deixa uma questão em aberto. A competência do sistema multiagente é medida de ponta a ponta , acurácia de triagem, acerto de causa raiz, sem que se isole a contribuição de cada fonte nem se caracterize seu regime de erro.

Table 1: Fases do CRISP-DM, notebooks e técnicas. As fases se repetem porque o processo é iterativo; a ordem segue a dos notebooks.

Fase	Notebook: conteúdo (técnica)
Negócio, Dados	01: problema, critérios de sucesso, qualidade e evolução mensal da base
Preparação	02: consolidação, atributos por janela, p95 de latência (percentil)
Dados, Modelagem	03: regimes (Kruskal-Wallis com Bonferroni; k-means com silhouette)
Modelagem	04: perfil contextual (mediana/MAD); limite por indicador (percentil, validação cruzada), por mês de teste e histórico
Preparação	05: cenários de incidente (injeção de falhas)
Avaliação	06: estado, indicador, início (expansão retroativa); histórico; IC de Wilson, McNemar
Implantação	07: consulta estruturada para o agente

2.5 Síntese e lacuna

A comparação revela um padrão: os trabalhos que dispensam rótulos tratam a normalidade como única ou como algo que se adapta ao longo do tempo, mas não como uma família de regimes legítimos coexistentes; os que modelam comportamento com riqueza suficiente para distinguir situações dependem de fontes além do AAL; e os que produzem explicações acionáveis o fazem no nível do sistema de agentes, assumindo como dadas as evidências que recebem. Nenhum deles investiga, exclusivamente a partir do AAL, se um mesmo recurso apresenta vários regimes normais e se essa caracterização pode informar, no instante do registro de um incidente, o estado do recurso, o indicador impactado e o início do desvio.

3 METODOLOGIA

O trabalho segue o CRISP-DM [**??**]. Cada fase foi implementada em um *notebook* Jupyter com parâmetros explícitos (Tabela **??**). A análise é sempre por endpoint, unidade executável dentro de um produto de API. Optou-se por seis técnicas de mineração de dados no total, todas disponíveis em bibliotecas Python de uso corrente, como pandas, SciPy e scikit-learn [**?**].

3.1 Entendimento do negócio

A decisão apoiada é: *no instante do registro do incidente, o recurso está em regime normal? Se não, qual indicador foi impactado (erros 4xx, erros 5xx, latência) e desde quando?* A abrangência (consumidor específico ou generalizado) é informação complementar. Os critérios de sucesso foram definidos antes da modelagem (Tabela **??**). Como não havia histórico de incidentes, a avaliação usa cenários simulados com falhas injetadas.

3.2 Entendimento dos dados

A base contém AALs de produção de 12 endpoints de APIs distintas, identificados aqui por E01 a E12 (em ordem decrescente de volume em agosto). Cada registro é um agregado por minuto de data hora da execução, endpoint, código de resposta HTTP (Status-code) e cliente com o total de execuções no minuto e o tempo médio dessas execuções. Como a agregação temporal por minuto já está disponível neste formato, optou-se por não utilizar os registros por execução, o que não inviabiliza a utilização do registro bruto para fazer essa agregação e utilizar como entrada para este processo. O identificador de cliente foi anonimizado por SHA-256 com semente, sem payloads nem dados pessoais. Os horários, em UTC, foram convertidos para o horário de Brasília (UTC-3) antes de qualquer corte temporal.

São 5.632.001 registros de 01/05/2026 0h a 19/09/2026 23h59 (horário de Brasília), com 1.976 clientes distintos. As respostas de sucesso (código de resposta HTTP = 2xx) somam 87,6% das transações; de erros negociais (código de resposta HTTP = 4xx), 12,1%; e erros de sistema (código de respostas HTTP = 5xx), 0,21%. A cobertura varia: sete recursos têm dados desde maio; o E04, desde 25/05; cinco (E02, E03, E06, E10, E11), apenas desde 16/07; e quatro (E02, E03, E06, E11) param em 03/09. O volume de execuções por 15 min vai de 47 transações (E11) a 2.239 (E02). Estes dados mostra a heterogeneidade da base de dados e por endpoint.

Evolução mensal. O comportamento muda ao longo dos meses. Entre maio e setembro, o volume médio diário cresce de 35% a 60% em vários recursos (E07, E08, E12), e a taxa de erros de negócio muda de patamar: no E02, passa de 32% (julho) para 45% (agosto) e 62% (setembro); no E04, de 0,5% (maio) para 13% (agosto). Essa mudança motiva a questão RQ4.

3.3 Preparação dos dados

As chaves repetidas foram consolidadas (soma do volume e média de latência ponderada). Os registros foram agregados em janelas de 1 h (29.695 janelas endpoint × hora), que formam a base do perfil de normalidade. A decisão usa quatro indicadores (Tabela **??**). O *contexto* de cada janela é o par (tipo de dia, hora), com tipo de dia *útil* ou *não útil* (sábado, domingo e feriados).

Latência p95. Como não há latência por requisição, o p95 é o percentil 95 das latências médias por minuto × cliente das respostas 2xx, ponderado pelo número de execuções. É um p95 *das médias*, não das requisições, e tende a subestimar a cauda real; mas é consistente entre períodos e sensível a degradações que atingem só parte dos minutos ou dos consumidores. Na mediana, ele é 1,42 vez a latência média.

3.4 Modelagem

Regimes (descritivo). O teste de Kruskal-Wallis [**?**], com correção de Bonferroni [**?**], verifica se cada indicador muda entre as 24 horas dos dias úteis. Para cada recurso, o k-means [**?**] agrupa as janelas de 1 h de maio a agosto usando apenas os indicadores padronizados e limitados aos percentis 1 e 99 do recurso (em um período de quatro meses, picos isolados formariam grupos próprios com menos de 0,5% das horas). O número de regimes, $k \in [2, 8]$, é escolhido pelo coeficiente de silhouette [**?**]. O contexto não entra no agrupamento, para que a relação regime–contexto seja uma descoberta, e não um insumo; ele é usado depois, para nomear cada regime (nível de volume, tipo de dia e horas em que predomina). O k-means é descritivo: a decisão usa o perfil contextual.

Table 2: Indicadores usados na decisão e direção desfavorável.

Indicador	Definição	Direção
volume	log(1+transações), na escala horária	alta e queda
erros 4xx	transações 4xx / total (erros de negócio)	alta
erros 5xx	transações 5xx / total (erros de sistema)	alta
latência p95	log(1+p95 das médias por minuto×cliente)	alta

Perfil de normalidade contextual. Para cada endpoint, contexto e indicador, o perfil guarda a mediana histórica e uma escala robusta de variação, calculadas nas janelas de uma hora do histórico. A escala é obtida a partir do desvio absoluto mediano (MAD), multiplicado por 1,4826 para ficar comparável ao desvio-padrão, e respeita um valor mínimo para evitar divisões por valores próximos de zero. Para cada período avaliado, o valor observado é comparado à mediana esperada para aquele endpoint e contexto, e essa diferença é dividida pela escala robusta. O resultado indica quantas escalas de variação normal o indicador se afasta do esperado. Para o volume, tanto aumentos como quedas são considerados desfavoráveis; para erros 4xx, erros 5xx e latência, apenas aumentos contam como desvio. Períodos de 15 e 30 minutos usam o mesmo perfil de uma hora: o volume é convertido para a escala horária, e as taxas e o p95 são usados diretamente.

Limite por indicador. Os indicadores têm variabilidades muito diferentes; em alguns endpoints a taxa de 4xx tem picos reais frequentes. Por isso, cada indicador tem um limite próprio no recurso, $L_{r,a} = \max\{3,\ Q_{0,99}(d_a)\}$: o percentil 99 dos seus desvios no histórico, calculados fora da amostra. O limite é recalculado para cada tamanho de período (15, 30 e 60 min), porque períodos curtos variam mais. O período é RESIDUAL se algum indicador passa do seu limite ($d_a/L_{r,a} > 1$); o **indicador responsável** é o que mais passou, em múltiplos do limite.

Histórico e calibração sem vazamento. Para cada mês de teste (agosto; 1 a 19 de setembro), o perfil e os limites são ajustados só com dados anteriores, em dois históricos: *3 meses* (principal; maio a julho para agosto e junho a agosto para setembro) e *1 mês* (o mês anterior). Os desvios fora

Table 3: Motivos injetados.

Motivo	Intensidade (baixa / média / alta / muito alta)
4xx generalizado	5% / 15% / 40% das 2xx viram 400
4xx de 1 consumidor	30% / 60% / 90% das 2xx do consumidor viram 400
5xx generalizado	5% / 15% / 40% das 2xx viram 503
5xx de 1 consumidor	30% / 60% / 90% das 2xx do consumidor viram 503
Latência	tempo de execução ×1,5 / ×2,5 / ×4 / ×10

O consumidor afetado é sorteado com probabilidade proporcional ao seu volume.

da amostra vêm de validação cruzada com quatro blocos contíguos de tempo dentro do histórico: o perfil é ajustado em três blocos e avalia o restante. Com essa regra, cerca de 3% dos períodos do histórico são RESIDUAL.

Consulta no registro e início estimado (expansão retroativa). No instante t do registro, o método olha para trás a partir de t em faixas disjuntas cada vez mais distantes: 0–15, 15–30 e 30–60 min e, depois, de hora em hora até 4 h. A faixa mais recente, $[t - 15\,\text{min}, t)$, define o *estado no registro* e o indicador responsável. Se ela for RESIDUAL, a busca continua para trás e para na primeira faixa NORMAL; o início é estimado entre o começo da última faixa residual e o fim dessa faixa normal ("começou entre 30 e 120 min antes do registro"). Todas as faixas são comparadas ao mesmo perfil de 1 h: o volume é convertido para a escala horária, e as taxas e o p95 são usados diretamente. Como faixas curtas variam mais, cada tamanho de faixa recebe seus próprios limites, calibrados da mesma forma.

Abrangência (complementar). Para erros 4xx ou 5xx, o desvio é atribuído a um *consumidor específico* quando o cliente com mais erros concentra ao menos 80% deles e essa fatia supera em pelo menos 20 pontos percentuais a sua fatia do volume; caso contrário, é *generalizado*. Recursos com menos de três clientes ativos ficam indeterminados.

3.5 Avaliação

Cenários de incidente. Cada cenário simula um incidente: uma degradação começa em t_0 e o incidente é registrado em $t = t_0 + \Delta$, com atraso $\Delta \in \{15, 30, 60, 120\}$ min. O instante de registro é sorteado dentro do mês de teste. Entram no teste de um mês os recursos com ao menos 28 dias de dados no mês anterior: 7 recursos em agosto e os 12 em setembro. As falhas (Tabela **??**) são aplicadas nos registros por minuto × cliente e depois reagregadas, para que os indicadores e a abrangência reajam como numa falha real. Foram gerados 5 cenários por recurso × mês × motivo × intensidade × atraso (6.080 incidentes) e 6.080 consultas de controle, em instantes sem falha injetada.

Latência ×10. Como a latência vem de médias por minuto, poucas requisições lentas elevam muito o valor. No histórico de maio a agosto, o p95 horário ultrapassou 10 vezes a mediana do seu contexto em 0,78% das horas (percentil 99,9: 25 vezes; máximo: 68 vezes). Degradações de latência relevantes para um incidente tendem a ser desse tamanho, por isso a intensidade "muito alta" (×10) foi incluída para a latência. As intensidades menores foram mantidas para mostrar o limite do método.

Métricas. Para cada cenário, a consulta é feita no instante do registro, como descrito acima. As métricas são: *sensibilidade* (incidentes com saída RESIDUAL); *especificidade* (controles com saída NORMAL); *acurácia do indicador* (indicador apontado igual ao do motivo injetado, entre os detectados); *acerto do início* (início real dentro da zona estimada pela expansão retroativa); e *acurácia da abrangência*. "Média/alta" refere-se às intensidades média e alta, sem a latência ×10, para comparação com versões anteriores do estudo. As proporções têm intervalos de confiança de Wilson a 95% [**?**].

Baselines. Os baselines avaliam se o perfil contextual agrega valor à decisão. O primeiro é o *z-score global*, que usa a mesma lógica de mediana, MAD e limites por indicador, mas ignora hora e tipo de dia: cada endpoint tem uma única referência histórica para cada indicador. O segundo é a *semana anterior*, que compara o período avaliado com o mesmo intervalo de tempo sete dias antes, explorando a periodicidade semanal sem construir um perfil estatístico mais amplo. Os três métodos usam o mesmo tratamento de limites e calibração, para que a comparação isole o efeito da estratégia de referência.

A comparação entre métodos usa o teste exato de McNemar [**?**] sobre os acertos de estado nos mesmos cenários simulados. O teste considera apenas os casos em que os métodos divergem: aqueles em que o perfil contextual acerta e o baseline erra, e aqueles em que ocorre o contrário. Um valor p pequeno indica evidência de diferença estatisticamente significativa entre os métodos. O mesmo teste compara os perfis ajustados com históricos de 3 meses e de 1 mês, pois ambos são aplicados aos mesmos cenários.

Experimentos. **Exp. 1** caracteriza os regimes de cada recurso (Kruskal-Wallis e k-means). **Exp. 2** avalia o estado no registro: sensibilidade, especificidade e comparação com os baselines. **Exp. 3** avalia a consulta quando o recurso é RESIDUAL: indicador responsável, início estimado pela expansão retroativa e abrangência. **Exp. 4** compara os históricos de 3 meses e de 1 mês.

Este trabalho mostrou que Access Logs agregados por minuto, sem *traces* nem logs de aplicação, permitem caracterizar os regimes normais de cada recurso de API e informar, no instante em que um incidente é aberto, se o recurso está em regime normal e, se não, qual indicador foi impactado (erros de negócio, erros de sistema ou latência) e desde quando. O método usa técnicas simples (mediana, MAD, percentis, validação cruzada e uma busca retroativa), não exige rótulos e produz uma explicação que um agente ou operador pode usar diretamente.

Em dois meses de teste, a informação foi confiável quando emitida: 97,1% de especificidade, 96,7% de acerto no indicador e 91,9% de acerto na zona de início. A sensibilidade ainda é limitada (39%): alta para erros de sistema, moderada para degradações grandes de latência e baixa para erros de negócio em recursos com muitos 4xx naturais. Com mais dados, o ganho de detecção do contexto sobre um perfil sem contexto não se confirmou; seu valor está na explicação. O histórico recente (1 mês) mostrou-se melhor que o longo (3 meses), o que recomenda retreino frequente.

Como trabalhos futuros, destacam-se: (i) validar o método com incidentes reais, começando pelo evento observado no E02, e com a avaliação de operadores; (ii) testar janelas de histórico deslizantes mais curtas e com pesos maiores para o período recente; (iii) melhorar a sensibilidade à latência com percentis por requisição; (iv) avaliar a generalização para outras APIs; e (v) integrar a consulta como ferramenta de um agente de apoio a incidentes.

REFERENCES

Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A. Zuluaga. 2020. USAD: UnSupervised Anomaly Detection on Multivariate Time Series. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20)*. ACM, New York.

Pete Chapman, Julian Clinton, Randy Kerber, Thomas Khabaza, Thomas Reinartz, Colin Shearer, and Rüdiger Wirth. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. Technical Report. SPSS.

Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, New York, 1285–1298.

Olive Jean Dunn. 1961. Multiple Comparisons among Means. *J. Amer. Statist. Assoc.* 56, 293 (1961), 52–64.

William H. Kruskal and W. Allen Wallis. 1952. Use of Ranks in One-Criterion Variance Analysis. *J. Amer. Statist. Assoc.* 47, 260 (1952), 583–621.

Stuart Lloyd. 1982. Least Squares Quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137.

Yu Luo et al. 2025. OpsAgent: An Evolving Multi-Agent System for Incident Management in Microservices. arXiv:2510.24145. Acesso em: 18 maio 2026.

Quinn McNemar. 1947. Note on the Sampling Error of the Difference between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157.

Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)*. IJCAI Organization, Macao, 4739–4745. https://doi.org/10.24963/ijcai.2019/658

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

Peter J. Rousseeuw. 1987. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65.

Gagan Somashekar, Anurag Dutt, Mainak Adak, Tania Lorido Botran, and Anshul Gandhi. 2024. GAMMA: Graph Neural Network-Based Multi-Bottleneck Localization for Microservices Applications. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*. ACM, New York, 3085–3095.

Yuqing Wang, Mika V. Mäntylä, Serge Demeyer, Mutlu Beyazit, Joanna Kisaakye, and Jesse Nyyssölä. 2025. Cross-System Categorization of Abnormal Traces in Microservice-Based Systems via Meta-Learning. *Proceedings of the ACM on Software Engineering* 2, FSE (2025). Artigo FSE027.

Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212.

Rüdiger Wirth and Jochen Hipp. 2000. CRISP-DM: Towards a Standard Process Model for Data Mining. In *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*. 29–39.

Zhaoyang Yu et al. 2025. Triangle: Empowering Incident Triage with Multi-Agent. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)*. IEEE/ACM. Disponível em: https://www.microsoft.com/en-us/research/wp-content/uploads/2025/02/TRIANGLE_ASE25.pdf. Acesso em: 18 maio 2026.

Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. DeepTraLog: Trace-Log Combined Microservice Anomaly Detection through Graph-Based Deep Learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, 623–634.

Wei Zhang et al. 2025. Intelligent Site Reliability Engineering: A Multi-Agent LLM Framework for Automated Incident Analysis and Root Cause Determination. https://www.researchgate.net/publication/399119176. Acesso em: 10 maio 2026.